hmseekr manual

Functions

kmers

Description:

This function counts k-mers for multiple specified values of k and saves them to a binary file. This function takes in a fasta file such as query seq or background seqs. If input fasta contains more than one sequence, all the sequences will be merged to one sequence before processing. It counts the kmer frequency for each kmer size in kvec (-k), kvec can also be a single kmer size. The output of the function is a dictionary of dictionaries with the outer dictionary keys as kmer size and the inner dictionary keys as kmer sequences and values as kmer counts.

This step should be performed for both query sequence and the background sequences, which then generate the .dict kmer profile for both query and background sequences.

Python Example:

generate kmer count files for kmer size 4 using mXist_rA.fa as input fasta file

```
from hmseekr.kmers import kmers
testdict = kmers(fadir='./fastaFiles/mXist_rA.fa',kvec='4',
                 alphabet='ATCG',outputname='repeatA',
                 outputdir='./counts/')
```

Console Example:

generate kmer count files for kmer size 2,3,4 using mXist_rA.fa as input fasta file

```
$ hmseekr_kmers -fd './fastaFiles/mXist_rA.fa' -k 2,3,4 -a ATCG -name
repeatA -dir './counts/'
```

Input:

fadir (-fd, --fadir): path to the input fasta file. If input fasta contains more than one sequence, all the sequences will be merged to one sequence before processing
kvec (-k, --kvec): Comma delimited string of possible k-mer values. For example, '3,4,5' or just '4'
alphabet (-a, --alphabet): Alphabet to generate k-mers, default=ATCG
outputname (-name, --outputname): name of output count file, default is 'out'
outputdir (-dir, --outputdir): path of output directory to save output count file, default is current directory

Output:

a dictionary of dictionaries with the outer dictionary keys as kmer size and the inner dictionary keys as kmer sequences and values as kmer counts
The output is saved as a binary file with .dict extension, which contains count matrices.

train

Description:

This step calculates the emission matrix based on kmer counts, prepare transition matrix, states and saves them to a binary file. This function takes in kmer count file for sequences of interest (e.g. query seq, functional regions of a lncRNA) and kmer count file for background sequences (e.g. null seq, transcriptome, genome). These kmer count files are generated using the kmers function with the k specified (which should be the same as calculted in the kmers function) and query to query transition rate (qT) and null to null transition rate (nT) specified.

It calculates the hidden state transition matrix, hidden state emission matrix, states and starting probability of each hidden state and saves them to a binary file.

Python Example:

train a model using previously generated kmer count files for repeatA and all lncRNA (kmers, or hmseekr_kmers function) with kmer size 2,3,4 and transition rates of 0.99 for both query to query and null to null, and save to the markovModels folder

```
from hmseekr.train import train


testmodel = train(querydir='./counts/repeatA.dict',
                  nulldir='./counts/all_lncRNA.dict',
                  kvec='2,3,4', alphabet='ATCG',
                  queryT=0.99, nullT=0.99,
                  queryPrefix='repeatA',
                  nullPrefix='lncNRA', outputdir='./markovModels/')
```

Console Example:

train a model using previously generated kmer count files for repeatA and all lncRNA (kmers, or hmseekr_kmers function) with kmer size 4 and transition rates of 0.9999 for both query to query and null to null. Save the model to the current directory.

```
$ hmseekr_train -qd './counts/repeatA.dict' -nd
'./counts/all_lncRNA.dict' -k 4 -a ATCG -qT 0.99 -nT 0.99 -qPre
repeatA -nPre lncRNA -dir './'
```

Input:

querydir (-qd, --querydir): Path to kmer count file for sequence of interest or query seq (e.g. functional regions of a ncRNA)
nulldir (-nd, --nulldir): Path to kmer count file that compose null model/background sequences (e.g. transcriptome, genome, etc.)
kvec (-k, --kvec): Comma delimited string of possible k-mer values. For example, '3,4,5' or just '4'. Numbers in kvec must be found in the k-mer count file (precalculated by kmers function)
alphabet (-a, --alphabet): Alphabet to generate k-mers, default=ATCG
queryT (-qT, --queryT): Probability of query to query transition, default=0.99, should be between 0 and 1 but not equal to 0 or 1
nullT (-nT, --nullT): Probability of null to null transition, default=0.93, should be between 0 and 1 but not equal to 0 or 1
queryPrefix (-qPre, --queryPrefix): prefix file name for query, defualt='query'
nullPrefix (-nPre, --nullPrefix): prefix file name for null, defualt='null'
outputdir (-dir, --outputdir): path of output directory to save output trained model file in .dict format, default is current directory

Output:

A dictionary containing models for each value of k specified. It saves hidden state transition matrix, hidden state emission matrix, states and starting probability of each hidden state. The output is saved as a binary file with .dict extension.

findhits_basic

Description:

This step uses precalculated model (emission matrix, prepared transition matrix, pi and states) from train (hmseekr_train) function to find out HMM state path through sequences of interest, therefore return sequences that has high similarity (based on kmer profile) to the query sequence -- hits sequneces. This function takes in a fasta file (searchpool) which defines the region to search for potential hits (highly similar regions to the query sequence), also takes in the precalculated model (train or hmseekr_train function). Along the searchpool fasta sequences, similarity scores to query sequence will be calculated based on the emission probabilities (E) established in the model, hits segments (highly similar regions) would be reported along with the sequence header from the input fasta file, start and end location of the hit segment, the actual sequence of the hit segment. It calculates the hidden state transition matrix, hidden state emission matrix, states and starting probability of each hidden state and saves them to a binary file.

Python Example:

use the previously trained model (hmm.dict by train/hmseekr_train function) to search for highly similar regions to query sequence (repeatA) within the pool.fa files (area of interest region to find sequences similar to repeatA, could be all lncRNAs or just

chromosome 6) with kmer size 4 and save the hit sequences while showing progress bar.

```
from hmseekr.findhits_basic import findhits_basic


testhits = findhits_basic(searchpool='../fastaFiles/pool.fa',
                          modeldir='../markovModels/hmm.dict',
                          knum=4,outputname='hits',outputdir='./',
                          alphabet='ATCG',fasta=True,progressbar=True)
```

Console Example:

use the previously trained model (hmm.dict by train/hmseekr_train function) to search for highly similar regions to query sequence (repeatA) within the pool.fa files (area of interest region to find sequences similar to repeatA, could be all lncRNAs or just chromosome 6) with kmer size 4 and save the hit sequences while showing progress bar.

```
$ hmseekr_findhits_basic -pool './fastaFiles/pool.fa' -m
'./markovModels/repeatA_lncRNA/4/hmm.dict' -k 4 -name 'hits' -dir './'
-a 'ATCG' -fa -pb
```

Input:

searchpool (-pool, --searchpool): Path to fasta file which defines the region to search for potential hits (highly similar regions) based on the precalculated model (train function)

modeldir (-m, --modeldir): Path to precalculated model .dict file output from train.py'

knum (-k, --knum): Value of k to use as an integer. Must be the same as the k value used in training (train function) that produced the model

outputname (-name, --outputname): File name for output, useful to include information about the experiment, default='hits'

outputdir (-dir, --outputdir): path of output directory to save output dataframe, default is current directory.

alphabet (-a, --alphabet): String, Alphabet to generate k-mers default='ATCG'

fasta (-fa, --fasta): whether to save sequence of hit in the output dataframe, default=True: save the actual sequences

progressbar (-pb, --progressbar): whether to show progress bar

Output:

A dataframe containing information about the hit regions: highly similar regions to query sequence based on the precalculated model within the input fasta file. Information about the hits regions includes: the sequence header from the input fasta file, start and end location of the hit segment, and the actual sequence of the hit segment if fasta=True.

findhits_condE

Description:

This is an advanced version of findhits_basic/hmseekr_findhits_basic function: it calculates the emission probability of the next word given the current word. In findhits_basic, as the shift is by 1 nt, the next word has k-1 overlap with the current word. For example, if the current word is 'TAGC', the next possible words are 'AGCA', 'AGCT', 'AGCC', 'AGCG'. Then the emission probability of the next word ('AGCA') given the current word as 'TAGC' is calculated as: np.log2(emission probability of 'AGCA' in the original E) - np.log2(sum of emission probability of all four possible worlds in the original E). findhits_condE is the default findhits function in higher level functions such as gridsearch or seqstosummary.

Python Example:

same example as findhits_basic but with the conditioned emission probability

```
from hmseekr.findhits_condE import findhits_condE


testhits = findhits_condE(searchpool='../fastaFiles/pool.fa',
                    modeldir='../markovModels/hmm.dict',
                    knum=4,outputname='hits',outputdir='./',
                    alphabet='ATCG',fasta=True,
                    progressbar=True)
```

Console Example:

same example as hmseekr_findhits_basic but with the conditioned emission

```
$ hmseekr_findhits_condE -pool './fastaFiles/pool.fa' -m
'./markovModels/repeatA_lncRNA/4/hmm.dict' -k 4 -name 'hits' -dir './'
-a 'ATCG' -fa -pb
```

Input and Output:

Inputs and Output are in the same format as findhits_basic/hmseekr_findhits_basic function

gridsearch

Description:

This function performs a grid search to find the best trasnition probabilities for query to query and null to null states, which is used in the train function. This function takes in query sequence, null squence and background sequence (see Inputs for details) fasta files, ranges and steps or values for query to query transition rate (qT) and null to null transition rate (nT), a specific kmer number and performs the train function and findhits_condE function for each combination of qT and nT. Within the hits sequences (findhits_condE function results), only keep the sequence with length greater than lenmin and less than lenmax. Then calculates pearson correlation r score (seekrPearson) between the filtered hit sequences (findhits_condE results) and the query sequence. It returns a dataframe (.csv file) containing the qT, nT, kmer number, the total number of hits sequences and the median, standard deviation of the hits sequences' pearson correlation r score to the query sequence and the median, standard deviation of the length of the hits sequences. Then if there are more than 50 hits, it calculates the same stats for the top 50 hits sequences, ranked by their seekr r score (seekrPearson); if there are less than 50 hits in total, the stats for the top 50 hits are the same as the stats for all the hits. If query fasta contains more than one sequence, all the sequences in query fasta file will be merged to one sequence, for calculating kmer count files for hmseekr (hmseekr.kmers) and for calculating seekrPearson. This function does not require the seekr package to be installed. As there are iterations of train and findhits_condE functions, which could take long time, it is recommended to run this function on a high-performance computing cluster. Variants of findhits functions can be specified to run.

Python Example:

perform a grid search to find the best transition probabilities for qT and nT each within the range of 0.9 to 0.99, and only keep the hit sequences with length greater than 100nt and less than 1000nt for stats calculation. the ‘findhits_basic’ function is used here.

```
from hmseekr.gridsearch import gridsearch


testsearch = gridsearch(queryfadir='./fastaFiles/repeatA.fa',
                        nullfadir='./fastaFiles/all_lncRNA.fa',
                        searchpool='./fastaFiles/pool.fa',
                        bkgfadir='./fastaFiles/bkg.fa',knum=4,
                        qTlist='0.9,0.99,0.01’, nTlist='0.9,0.99,0.01’,
                        stepmode=True, func='findhits_basic',
                        lenmin=100,lenmax=1000,
                        outputname='gridsearch_results',
                        outputdir='./gridsearch/',
                        alphabet='ATCG', progressbar=True)
```

perform a grid search to find the best transition probabilities for qT and nT each exactly as 0.9,0.99,0.999, and only keep the hit sequences with length greater than 100nt and less than 1000nt for stats calculation. the conditioned Emission ‘findhits_condE’ function is used here.

```
from hmseekr.gridsearch import gridsearch


testsearch = gridsearch(queryfadir='./fastaFiles/repeatA.fa',
                    nullfadir='./fastaFiles/all_lncRNA.fa',
                    searchpool='./fastaFiles/pool.fa',
                    bkgfadir='./fastaFiles/bkg.fa',knum=4,
                    qTlist='0.9,0.99,0.999',nTlist='0.9,0.99,0.999',
                    stepmode=False, func='findhits_condE',
                    lenmin=100, lenmax=1000,
                    outputname='gridsearch_results',
                    outputdir='./gridsearch/',
                    alphabet='ATCG', progressbar=True)
```

Console Example:

perform a grid search to find the best transition probabilities for qT and nT each within the range of 0.9 to 0.99, and only keep the hit sequences with length greater than 100nt and less than 1000nt for stats calculation. the findhits_basic function is used here.

```
$ hmseekr_gridsearch -qf './fastaFiles/repeatA.fa' -nf
'./fastaFiles/all_lncRNA.fa' -pool './fastaFiles/pool.fa' -bkgf
'./fastaFiles/bkg.fa' -k 4 -ql 0.9,0.99,0.01 -nl 0.9,0.99,0.01 -step -
fc findhits_basic -li 100 -la 1000 -name 'gridsearch_results' -dir
'./gridsearch/' -a 'ATCG' -pb
```

perform a grid search to find the best transition probabilities for qT and nT each exactly as 0.9,0.99,0.999, and only keep the hit sequences with length greater than 100nt and less than 1000nt for stats calculation. the condirioned Emission ‘findhits_condE’ function is used here.

```
$ hmseekr_gridsearch -qf './fastaFiles/repeatA.fa' -nf
'./fastaFiles/all_lncRNA.fa' -pool './fastaFiles/pool.fa' -bkgf
'./fastaFiles/bkg.fa' -k 4 -ql 0.9,0.99,0.999 -nl 0.9,0.99,0.999 -fc
findhits_condE -li 100 -la 1000 -name 'gridsearch_results' -dir
'./gridsearch/' -a 'ATCG' -pb
```

Input:

queryfadir (-qf, --queryfadir): Path to the fasta file of query seq (e.g. functional regions of a ncRNA). If query fasta contains more than one sequence, all the sequences in query

fasta file will be merged to one sequence for calculating seekrPearson and for calculating kmer count files for hmseekr

nullfadir (-nf, --nullfadir): Path to the fasta file of null model sequences (e.g. transcriptome, genome, etc.)

searchpool (-pool, --searchpool): Path to fasta file which defines the region to search for potential hits (highly similar regions) based on the precalculated model (train function)

bkgfadir (-bkgf, --bkgfadir): fasta file directory for background sequences, which serves as the normalizing factor for the input of seekr_norm_vectors and used by seekr_kmer_counts function. This fasta file can be different from the nullfadir fasta file

knum (-k, --knum): a single integer value for kmer number

qTlist (-ql, --qTlist): specify probability of query to query transition. if stepmode is False, the input should be a string of numbers separated by commas: '0.9,0.99,0.999' with no limit on the length of the list. all the numbers in the list that are greater than 0 and less than 1 are used as qT values in the iteration. if stepmode is True, the input should be a string of exactly three numbers separated by commas: '0.1,0.9,0.05' as min, max, step. the min, max, step values are used to generate a list of qT values, with min and max included. all the numbers that are greater than 0 and less than 1 are used as qT values in the iteration.

nTlist (-nl, --nTlist): specify probability of null to null transition. the setting is the same as in qTlist.

stepmode (-step, --stepmode): True or False, defines whether to use the qTlist and nTlist as min, max, step or as a list of values for qT and nT. Default is True: use qTlist and nTlist as min, max, step.

func (-fc, --func): the function to use for finding hits, default='findhits_condE', the other option is 'findhits_basic'

lenmin (-li; --lenmin): keep hits sequences that have length > lenmin for calculating stats in the output, default=100.

lenmax (-la; --lenmax): keep hits sequences that have length < lenmax for calculating stats in the output, default=1000.

outputname (-name, --outputname): File name for output dataframe, default='gridsearch_results'

outputdir (-dir, --outputdir): path of output directory to save outputs and intermediate files, default is a subfolder called gridsearch under current directory. The intermediate fasta seq files, count files, trained models and hits files are automatically saved under the outputdir into subfolders: seqs, counts, models, hits, where qT and nT are included in the file names as the iterated transition probabilities

alphabet (-a, --alphabet): String, Alphabet to generate k-mers default='ATCG'

progressbar (-pb, --progressbar): whether to show progress bar, default=True: show progress bar

Output:

a dataframe containing information about qT, nT, kmer number, the total number of hits sequences and the median, standard deviation of the hits sequences' pearson correlation r score to the query sequence and the median, standard deviation of the length of the hits sequences, and the same stats for the top 50 hits sequences (ranked by seekr r score) if there are more than 50 hits, after filtering by lenmin and lenmax

hitseekr

Description:

This function takes in the output of either findhits function and the query sequence fasta file with a background fasta file, and calculates the seekr pearson correlation r and p value between the hit sequences and the query sequence. It fit the background sequences to the common10 distributions and takes the best ranked distribution. It calculates the seekr pearson correlation r and p values between the hit sequences and the query sequence based on the best ranked distribution. It adds the seekr r and p value to the hits dataframe. The seekr r and p value could provide additional information about the similarity between the hit sequences and the query sequence.

Python Example:

add seekr p value to the hits dataframe generated by either findhits function, keeping hits with length between 100nt and 1000nt and have seekr p values less than 0.5 and r values greater than 0

```
from hmseekr.hitseekr import hitseekr

addpvals = hitseekr(hitsdir='./mm10_queryA_4_viterbi.txt',
                    queryfadir='./fastaFiles/mXist_rA.fa',
                    bkgfadir='./all_lncRNA.fa',
                    knum=4, lenmin=100, lenmax=1000, pfilter=0.5,
                    rfilter=0, outputdir='./', progressbar=True)
```

Console Example:

add seekr p value to the hits dataframe generated by either findhits function, keeping hits with length between 100nt and 1000nt and have seekr p values less than 0.5 and r values greater than 0

```
$ hmseekr_hitseekr -hd './mm10_queryA_4_viterbi.txt' -qf
'./fastaFiles/repeatA.fa' -bkgf './fastaFiles/bkg.fa' -k 4 -li 100 -la
1000 -pf 0.5 -rf 0 -dir './' -pb
```

Input:

hitsdir (-hd, --hitsdir): Path to the directory of the .txt file generated by either findhits function

queryfadir (-qf, --queryfadir): Path to the fasta file of query seq (e.g. functional regions of a ncRNA). If query fasta contains more than one sequence, all the sequences in query fasta file will be merged to one sequence for calculating seekrPearson and for calculating kmer count files for hmseekr

bkgfadir (-bkgf, --bkgfadir): fasta file directory for background sequences, which serves as the normalizing factor for the input of seekr_norm_vectors and used by seekr_kmer_counts function. This fasta file can be different from the nullfadir fasta file

knum (-k, --knum): a single integer value for kmer number

lenmin (-li; --lenmin): only keep hits sequences that have length > lenmin for calculating stats in the output, default=100. if no filter is needed, set to 0

lenmax (-la; --lenmax): only keep hits sequences that have length < lenmax for calculating stats in the output, default=1000=, if no filter is needed, set to a super large number

pfilter (-pf, --pfilter): only keep hits sequences that have seekr pearson correlation p value < pfilter for calculating stats in the output, default=1.1. if no filter is needed, set to 1.1

rfilter (-rf, --rfilter): only keep hits sequences that have seekr pearson correlation r value > rfilter for calculating stats in the output, default=-1.1. if no filter is needed, set to -1.1

outputdir (-dir, --outputdir): path of output directory to save outputs and intermediate files, default is a subfolder called gridsearch under current directory. The intermediate fasta seq files, count files, trained models and hits files are automatically saved under the outputdir into subfolders: seqs, counts, models, hits, where qT and nT are included in the file names as the iterated transition probabilities

progressbar (-pb, --progressbar): whether to show progress bar, default=True: show progress bar

Output:

a dataframe with seekr r and p value added to the findhits dataframe. outputname is automatically generated as the input findhits filename with '_seekr' appended to it

seqstosummary

Description:

This function is designed to get the overall likeliness of each search pool sequence to the query sequences. The function takes in a fasta file of multiple query sequences, a transtion probabilty dataframe, a search pool fasta file, a null fasta file (for hmseekr) and a background fasta file (for seekr). Here the transition probability dataframe must have the same rows as the query fasta file. The columns should be [qT,nT] where qT is the probability of query to query transition, nT is the probability of null to null transition The transition prbability for each query sequence can be different and can be optimized by the gridsearch function. Please include 'qT' and 'nT' as the first row (the column names) for the two columns in the csv file. Please check the transdf.csv file in the Github repository (https://github.com/CalabreseLab/hmseekr) for an example. Users can also choose to set the transition probability to be the same for all query sequences. The length filter file should have the same rows as the query fasta file, the columns should be '[lenmin,lenmax]' where lenmin is the minimum length of the hit region to keep (>lenmin) and lenmax is the maximum length of the hit region to keep (<lenmax). Please include 'lenmin' and 'lenmax' as the first row (the column names) for the two columns in the csv file. The length filter for each query sequence can be different based on the length of the query sequence. Please check the lenfilter.csv file in the repository (https://github.com/CalabreseLab/hmseekr) for an example. Make sure the order of the rows in the transition probability dataframe and length filter file matches the order of the query sequences in the fasta file. The function will run the kmers, train, findhits_condE and hitseekr functions for each query sequence. Then the results can be filtered by the length of the hit regions, and the seekr pearson correlation p value. Finally for each search pool sequence, the function will calculate the counts of filtered hit regions with a specific query sequence, and also seekr pval, and the minimal seekr pval for all the counts of a search pool sequence with each the query sequences. For long format each row of the output dataframe contains a sequence in the search pool fasta, and has eight columns: seqName, feature, counts, len_sum, pval_median, pval_min: seqName corresponds to the header in the search pool fasta file; feature corresponds to the header in the query fasta file; counts is the counts of filtered hit regions of the search pool sequences with the query sequences; len_sum is the sum of the length of all counts of a search pool sequence with the query sequences; pval_median is the median of seekr pearson correlation p value for each search pool sequence with the query sequences; pval_min is the minimum of seekr pearson correlation p value for each search pool sequence with the query sequences. For wide format: each row of the output dataframe corresponds to a sequence in the search pool fasta, and columns are eachfeature_counts, eachfeature_len_sum, eachfeature_pval_median. Wide format has the unique column (not included in the long format) that summarize the overall likeliness of each search pool sequence to all the query sequences. This stat is listed under the column name 'unique_coverage_fraction', which is the fraction of the total length of the search pool sequence that is covered by the unique hit regions across all the query sequences. Overlapping hit regions are merged and only the unique regions are

counted here. It also includes columns for the total length of each pool seq (seq_total_length) and the total length of the unique hit regions across all the query sequences (total_unique_coverage). The output dataframe can then be used to generalize an overall likeliness of each search pool sequence to all the query sequences.

Python Example:

search all genes on chr16 for the potential hit counts and similarities to the query sequences include mXist repeat A, B, C and E, filtering and keep hit sqeuences with length filter provided in lenfilter, and p val less than 0.5 for stats calculation. the conditioned emission findhits_condE function is used, print the output in wide format

```
from hmseekr.seqstosummary import seqstosummary


testsum = seqstosummary(queryfadir='./mXist_repeats.fa',
                        transdf='./transdf.csv',
                                        lenfilter='./lenfilter.csv',
                        nullfadir='./fastaFiles/expressed_lncRNA.fa',
                        searchpool='./fastaFiles/pool.fa',
                        bkgfadir='./fastaFiles/all_lncRNA.fa',
                        knum=4, func='findhits_condE',
                        pfilter=1,
                        outputname='seqstosummary_results',
                        outputdir='/Users/tsummary/',outdfformat='wide',
                        alphabet='ATCG', progressbar=True)
```

Console Example:

search all genes on chr16 for the potential hit counts and similarities to the query sequences include mXist repeat A, B, C and E, filtering and keep hit sqeuences with length filter provided in lenfilter, and p val less than 0.5 for stats calculation. the conditioned emission findhits_condE function is used, print the output in both long and wide format

```
$ hmseekr_seqstosummary -qf './fastaFiles/mXist_repeats.fa' -td
'./transdf.csv' -lf './lenfilter.csv' -nf './fastaFiles/all_lncRNA.fa'
-pool './fastaFiles/chr16.fa' -bkgf './fastaFiles/bkg.fa' -k 4 -fc
'findhits_condE' -pf 1 -name 'seqstosummary_results' -dir
'./seqstosummary/' -format both -a 'ATCG' -pb
```

Input:

queryfadir (-qf, --queryfadir): Path to the fasta file of query seqs. Different from other functions such as kmers and gridsearch, if query fasta contains more than one sequence, each sequence will be treated as a separate query sequence

transdf (-td, --transdf): Path to the transition probability dataframe in csv format, the dataframe should have the same rows as the query fasta file, and the columns should be [qT,nT] where qT is the probability of query to query transition, nT is the probability of null to null transition. Please include 'qT' and 'nT' as the first row (the column names) for the two columns in the csv file. Please do not include the index column in the csv file, but make sure the order of the rows in the csv file matches the order of the query sequences in the fasta file.

lenfilter (-lf; --lenfilter): Path to the length filter csv file, the dataframe should have the same rows as the query fasta file, and the columns should be [lenmin,lenmax] where lenmin is the minimum length of the hit region to keep (>lenmin) and lenmax is the maximum length of the hit region to keep (<lenmax). Please do not include the index column in the csv file, but make sure the order of the rows in the csv file matches the order of the query sequences in the fasta file.

nullfadir (-nf, --nullfadir): Path to the fasta file of null model sequences (e.g. transcriptome, genome, etc.)

searchpool (-pool, --searchpool): Path to fasta file which defines the region to search for potential hits (highly similar regions) based on the precalculated model (train function)

bkgfadir (-bkgf, --bkgfadir): fasta file directory for background sequences, which serves as the normalizing factor for the input of seekr_norm_vectors and used by seekr_kmer_counts function. This fasta file can be different from the nullfadir fasta file

knum (-k, --knum): a single integer value for kmer number

func (-fc, --func): the function to use for finding hits, default='findhits_condE', the other option is 'findhits_basic'

lenmin (-li; --lenmin): only keep hits sequences that have length > lenmin for calculating stats in the output, default=100. if no filter is needed, set to 0

lenmax (-la; --lenmax): only keep hits sequences that have length < lenmax for calculating stats in the output, default=1000.

pfilter (-pf, --pfilter): only keep hits sequences that have seekr pearson correlation p value < pfilter for calculating stats in the output, default=1.1. if no filter is needed, set to 1.1

outputname (-name, --outputname): File name for output dataframe, default= seqstosummary_results '

outputdir (-dir, --outputdir): path of output directory to save outputs and intermediate files, default is a subfolder called seqstosummary under current directory. The intermediate fasta seq files, count files, trained models and hits files are automatically saved under the outputdir into subfolders: seqs, counts, models, hits, where qT and nT are included in the file names

outdfformat (-format, --outdfformat): the format of the output dataframe, default='both', where both long and wide format would be saved, other options are 'wide' or 'long'

alphabet (-a, --alphabet): String, Alphabet to generate k-mers default='ATCG'

progressbar (-pb, --progressbar): whether to show progress bar, default=True: show progress bar

Output:

a dataframe in long format: where each row contains a sequence in the search pool fasta file, and eight columns: seqName, feature, counts, len_sum, pval_median, pval_min: seqName corresponds to the header in the search pool fasta file; feature corresponds to the header in the query fasta file; counts is the counts of filtered hit regions of the search pool sequences with the query sequences; len_sum is the sum of the length of all counts of a search pool sequence with the query sequences; pval_median is the median of seekr pearson correlation p value for each search pool sequence with the query sequences; pval_min is the minimum of seekr pearson correlation p value for each search pool sequence with the query sequences. In wide format: each row of the output dataframe corresponds to a sequence in the search pool fasta, and columns are eachfeature_counts, eachfeature_len_sum, eachfeature_pval_median, eachfeature_pval_min. Wide format has the unique column (not included in the long format) that summarize the overall likeliness of each search pool sequence to all the query sequences. This stat is listed under the column name 'unique_coverage_fraction', which is the fraction of the total length of the search pool sequence that is covered by the unique hit regions across all the query sequences. Overlapping hit regions are merged and only the unique regions are counted here. It also includes columns for the total length of each pool seq (seq_total_length) and the total length of the unique hit regions across all the query sequences (total_unique_coverage).

hmseekr Docker Image

We now provide a Docker image of hmseekr which hopefully avoids all the package dependencies and complications when install and run hmseekr through pip.

Docker Installation

Firstly, you should install Docker on your computer. Please refer to the official website (https://www.docker.com/get-started/) for installation instructions.

After you have successfully installed Docker. Start/Run the application and make sure it is running properly – you should see the docker icon on your task bar with the status indicated.

Pull Docker Image and Test Run

1. Start your command line tool: Terminal for MacOS and CMD for Windows. You can also use Powershell or Cygwin for Windows, but Cygwin might have interaction issues.

From the command line, pull the Docker Image:

```
$ docker pull calabreselab/hmseekr:latest
```

You can replace `latest` with a specific tag if needed.

2. Test Run the Docker Image

```
$ docker run -it --rm calabreselab/hmseekr:latest
```

The `-it` tag sets it to interactive mode. If you don't need to run the Docker container in interactive mode (i.e., you don't need a shell inside the container), you can simply omit the `-it` flag.

This will print the user manual out to the command line, which is basically the same as you run the command `hmseekr` directly from command line when you pip install seekr.

Run Docker Image from command line

You can run the seekr function from this Docker Image directly from command line with the following specified syntax.

```
$ docker run -v /path/to/your/files:/data calabreselab/hmseekr:latest
hmseekr_kmers -fd '/data/fastaFiles/mXist_rA.fa' -k 2,3,4 -a ATCG -
name repeatA -dir '/data/counts/'
```

In this command:

* `-v /path/to/your/files:/data`: This mounts the directory `/path/to/your/files` from your host machine (where /fastaFiles/mXist_rA.fa is located) to the `/data` directory inside the Docker container. Replace `/path/to/your/files` with the actual path to your files.

* `mseekr_kmers -fd '/data/fastaFiles/mXist_rA.fa' -k 2,3,4 -a ATCG -name repeatA -dir '/data/counts/'`: This is the command that gets executed inside the Docker container. Since we mounted our files to `/data` in the container, we reference them with `/data/fastaFiles/mXist_rA.fa`.

* The `/data` folder is basically a mirror of the folder you specified in `/path/to/your/files`. So by specifying `-dir '/data/counts/'` (output into the counts folder under the path /data/) we can have the output files directly written in to the folder in `/path/to/your/files`.

* Please remember to **specify your output path to `/data/`** otherwise it will not be saved to your folder on local machine and it would be hard to locate it even inside the

Docker Container Filesystem (in this case, when the Docker Container is removed, your data will be deleted as well).

Examples of code mounts e:/test on Windows as the folder that contains the input and holds the output files:

```
$ docker run -v e:/test:/data calabreselab/hmseekr:latest
hmseekr_kmers -fd '/data/fastaFiles/mXist_rA.fa' -k 2,3,4 -a ATCG -
name repeatA -dir '/data/counts/'
```

Basically you need to add: `docker run -v /path/to/your/files:/data calabreselab/hmseekr:latest` before the command line code for hmseekr (see above for examples of all functions).

Run Docker Image with Jupyter Notebook

If you want to work with python codes instead of directly calling from the command line, you can choose to run seekr with Jupyter Notebook inside the Docker Container.

1. Run Docker Container with Interactive Terminal:

```
$ docker run -it -p 8888:8888 -v /path/on/host:/path/in/container
calabreselab/hmseekr:latest /bin/bash
```

This command will start the container and give you a bash terminal inside it. The `-p 8888:8888` flag maps the port *8888* from the container to your host machine so that you can access the Jupyter Notebook server.

`/path/on/host` is the path to a directory on your local machine where you want the notebooks and other data to be saved. `/path/in/container` is the directory inside the container where you want to access and save the files from Jupyter Notebook.

When you use Jupyter Notebook now and create or save notebooks, they will be stored at `/path/in/container` inside the Docker container. Due to the volume mount, these files will also be accessible and stored at `/path/on/host` on your host machine so that you can later access the code and files even when the container is removed.

Example of code:

```
$ docker run -it -p 8888:8888 -v e:/test:/data
calabreselab/hmseekr:latest /bin/bash
```

2. Manually start Jupyter Notebook. From the bash terminal inside the Docker container:

```
$ jupyter notebook --ip=0.0.0.0 --port=8888 --NotebookApp.token='' --
NotebookApp.password='' --allow-root
```

* `--ip=0.0.0.0`: Allow connections from any IP address. This is important for accessing Jupyter from outside the container.

* `--port=8888`: Run Jupyter on this port. You can change this if needed, but remember to adjust the `-p` flag when starting the Docker container.

* `--allow-root`: Allow Jupyter to be run as the root user inside the Docker container.

* `--NotebookApp.token="`: This disables the token-based security measure.

* `--NotebookApp.password="`: This ensures that there's no password required to access the Jupyter server.

Disabling the token and password for Jupyter Notebook reduces security. It's generally okay for local development, but you should avoid doing this in production or any publicly accessible server.

3. Access the Jupyter Notebook from your host machine's browser by entering this address:

http://localhost:8888/

4. Run Python functions as demonstrated above. It would be convenient if all input files can be copied over to the folder you have mounted: `/path/on/host`. Pay attention that when you specify your input and output file route, use the `/path/in/container` route, as that is a mirror to your local folder.

Example of code:

```
from hmseekr.kmers import kmers



testdict = kmers(fadir='/data/fastaFiles/mXist_rA.fa',kvec='4',

                alphabet='ATCG',outputname='repeatA',

                outputdir='/data/counts/')
```

Once you are done, you can click the **Shut Down** button under the **File** tab of Jupyter Notebook to shut down the instance or you can just click `Ctrl+C` twice from the command line to kill the process.

Then you need to exit the Docker Container from the interative session by typing `exit` and hit enter.

Cleanup (Optional)

* Clean Up Docker Container:
    + List all containers, including the stopped ones: `docker ps -a`
    + To remove a specific container: `docker rm CONTAINER_ID_OR_NAME`
    + To remove all stopped containers: `docker container prune`
* Clean Up Docker Image. If you want to remove the Docker image you used:
    + List all images: `docker images`
    + Remove a specific image: `docker rmi IMAGE_ID_OR_NAME`

You'd typically only remove an image if you're sure you won't be using it again soon, or if you want to fetch a fresh version of it from a repository like Docker Hub.

* Additional Cleanup. Docker also maintains a cache of intermediate images and volumes. Over time, these can accumulate. To free up space:

    + Remove unused data: `docker system prune`

    + To also remove unused volumes (be careful, as this might remove data you want to keep): `docker system prune --volumes`

Remember to always be cautious when cleaning up, especially with commands that remove data. Make sure you have backups of any essential data, and always double-check what you're deleting.

Issues and questions

For more details of the inputs and outputs, please refer to the manual listed under https://github.com/CalabreseLab/hmseekr/

Any issues can be reported to https://github.com/CalabreseLab/hmseekr/issues

Contact the authors at: shuang9@email.unc.edu; mauro_calabrese@med.unc.edu